Reinforcement Learning for DC Motor Speed Control

Code Assignment Report

Submitted by

Mazen Younis – 952106
Mert Satici – 951498

Faculty Department VI – Computer Engineering
Berlin University of Applied Sciences and Technology

Submission Date: 3 February 2026

Supervisors
Prof. Dr.-Ing. Francisco Morales Serrano Berliner Hochschule für Technik

BHT Berliner Hochschule für Technik

Studiere Zukunft

Table of Contents

1 Introduction **1**

1.1 Task and performance requirements 1
1.2 Why reinforcement learning? 1
1.3 Solution overview 1
1.4 Report structure 2

2 Problem Definition and Objectives **2**

2.1 Control objective 3
2.2 System inputs, outputs, and disturbances 3
2.3 Performance requirements 3
2.4 Reinforcement learning formulation 4
2.5 Summary 4

3 Environment Modeling **4**

3.1 DC motor dynamics 5
3.2 Load modeling 5
3.3 Discrete-time simulation 5
3.4 State, action, and episode definition 6
3.5 Environment outputs and diagnostics 6
3.6 Open-loop environment behavior 6

4 Reinforcement Learning Formulation **6**

4.1 Markov Decision Process formulation 7
4.2 Control objective 7
4.3 SARSA learning algorithm 8
4.4 Exploration strategy 8
4.5 Tabular representation and evaluation 8

5 Reward Function Design **8**

5.1 Design objectives 8
5.2 Reward definition 9
5.3 Tracking and control penalties 9
5.4 Progress shaping 9
5.5 Overshoot and wrong-direction penalties 10
5.6 Early termination and success bonus 10
5.7 Reward weights 10

6 Test Scenarios and Experimental Setup **10**

6.1 Load scenarios 10
6.2 Open-loop baseline 11
6.3 Training and evaluation protocol 11
6.4 Performance metrics 12

6.5 Reproducibility 12

7 Training Procedure and Hyperparameters 12
7.1 Training procedure 12
7.2 Episode termination 13
7.3 Hyperparameters 13
7.4 Learning curves 13
7.5 Extended training 15
7.6 Implementation notes 17

8 Results and Performance Evaluation 17
8.1 Closed-loop speed tracking 17
8.2 Quantitative performance metrics 19
8.3 Control signal behavior 20

9 Discussion 22
9.1 Tracking performance and robustness 22
9.2 Role of reward shaping 22
9.3 Control signal characteristics 22
9.4 Algorithm choice and limitations 23
9.5 Comparison to classical control 23
9.6 Possible improvements 23
9.7 Summary 23

10 Conclusion 24

List of Figures

Fig. 1 Open-loop speed response under a constant voltage command (V = 3.0 V) for different load scenarios. 11
Fig. 2 Training progress under nominal load (600 episodes). 14
Fig. 3 Training progress under no load (600 episodes). 14
Fig. 4 Training progress under high load (600 episodes). 15
Fig. 5 Training progress under nominal load (1000 episodes). 16
Fig. 6 Training progress under high load (1000 episodes). 16
Fig. 7 Training progress under no load (1000 episodes). 17
Fig. 8 Closed-loop speed response under no-load, nominal-load, and high-load conditions. 18
Fig. 9 Speed response under no-load condition. 18
Fig. 10 Speed response under nominal-load condition. 19
Fig. 11 Speed response under high-load condition. 19
Fig. 12 Greedy control signal under no load. 20
Fig. 13 Greedy control signal under nominal load. 21
Fig. 14 Greedy control signal under high load. 21

List of Tables

Tab. 2 SARSA training hyperparameters 13
Tab. 3 Closed-loop performance metrics of the greedy SARSA policy. 20

Symbols and Abbreviations

Symbol / Abbreviation	Unit	Description
MDP	–	Markov Decision Process
RL	–	Reinforcement Learning
SARSA	–	State–Action–Reward–State–Action learning algorithm
a_k	V	Voltage action applied at discrete time step k
$e(t)$	rad/s	Speed tracking error, $e(t) = \omega_{\text{ref}} - \omega(t)$
e_{ss}	rad/s	Steady-state speed tracking error
$e_k^{(b)}$	–	Discretized speed error bin (unitless index)
G	–	Episode return (cumulative reward over one episode)
G_k	–	Discounted return starting at time step k
$i(t)$	A	Armature current of the DC motor
J	kg·m^2	Rotor moment of inertia
k	–	Discrete-time index
k_e	V·s/rad	Back electromotive force (EMF) constant
k_t	N·m/A	Motor torque constant
L	H	Armature inductance
$\mathcal{A}$	–	Discrete action set / finite set of admissible voltage commands
$\mathcal{M}$	–	MDP model tuple (Markov Decision Process formulation)
$\mathcal{S}$	–	State space
M_p	%	Maximum overshoot relative to ω_{ref}
N_a	–	Number of discrete actions
N_e	–	Number of error discretization bins
$N_{\dot{e}}$	–	Number of error-derivative discretization bins
N_ω	–	Number of angular speed discretization bins
P	–	State transition probability function
$\pi(a \mid s)$	–	Control policy mapping states to actions
$Q(s, a)$	–	State–action value function
R	Ω	Armature resistance (motor parameter)
$\mathcal{R}(s, a)$	–	Reward function (MDP component)
r_{k+1}	–	Scalar reward received after transition from (s_k, a_k)
$r_{e,k}$	–	Tracking error penalty term
$r_{p,k}$	–	Progress shaping reward term
$r_{u,k}$	–	Control effort penalty term
$r_{\Delta u,k}$	–	Control smoothness (voltage change) penalty term

Symbol / Abbreviation	Unit	Description
$r_{\text{overshoot}}$	–	Overshoot penalty term
r_{wrong}	–	Wrong-direction control penalty term
$s(t)$	–	Continuous state vector of the environment
s_k	–	Discretized state at time step k
t_{90}	ms	Rise time to reach 90% of ω_{ref}
$\tau(t)$	N·m	Electromagnetic torque generated by the motor
τ_{load}	N·m	External load torque disturbance
Δt	s	Discrete simulation time step
$\dot{e}(t)$	rad/s^2	Time derivative of the tracking error
$\dot{e}_k^{(b)}$	–	Discretized error derivative bin (unitless index)
u_k	V	Control input (voltage) at time step k
u_{k-1}	V	Control input (voltage) at time step $k-1$
V	V	Motor terminal voltage (control input)
$V(t)$	V	Time-dependent motor voltage command
V_{max}	V	Maximum allowable motor voltage magnitude
ω	rad/s	Angular speed of the DC motor
$\omega(t)$	rad/s	Time-dependent angular speed of the motor
ω_{ref}	rad/s	Reference angular speed to be tracked
$\omega_k^{(b)}$	–	Discretized angular speed bin (unitless index)
α	–	Learning rate (SARSA step size)
γ	–	Discount factor weighting future rewards
ϵ	–	Exploration probability in the ϵ-greedy policy
ϵ_0	–	Initial exploration rate at the start of training
ϵ_{min}	–	Minimum exploration rate during training
$\tilde{e}_k$	–	Normalized speed tracking error
$\tilde{u}_k$	–	Normalized control input
$\tilde{\Delta u}_k$	–	Normalized control input change
w_e	–	Weight of tracking error penalty
w_o	–	Overshoot penalty weight
w_p	–	Weight of progress shaping reward
w_u	–	Weight of control effort penalty
$w_{\Delta u}$	–	Weight of control smoothness penalty
w_w	–	Weight of wrong-direction control penalty

1 Introduction

Electric DC motors are widely used in robotics and automation because they are compact, efficient, and easy to actuate via voltage commands. In this work, we consider the **Maxon DCX 32 L (graphite brushes)** motor and aim to regulate its angular speed to a fixed reference using **reinforcement learning (RL)**.

1.1 Task and performance requirements

The control objective is to track a constant speed reference ω_{ref} while meeting strict transient constraints:

- **Rise time:** $t_r \leq 1$ ms (time to reach 90% of ω_{ref}).
- **Overshoot:** $M_p \leq 3\%$ relative to ω_{ref}.
- **Control input:** motor voltage V, subject to saturation $|V| \leq V_{\text{max}}$.

Additionally, the learned controller must remain effective under multiple **load torque scenarios** (no-load, nominal load, and high load), which are used as test cases during evaluation.

1.2 Why reinforcement learning?

A classical approach to speed control is PI/PID tuning (or model-based design such as pole placement), which can work well but often requires:

- accurate modeling and parameter identification,
- retuning when load conditions change,
- additional logic to satisfy constraints (e.g., overshoot limits) and to avoid aggressive switching.

RL offers an alternative by learning a feedback policy directly from interaction with the environment and incorporating design goals and constraints through a shaped reward. This is especially useful when we want a **single policy** that generalizes across different load conditions without manual retuning.

1.3 Solution overview

We formulate the speed-control task as a finite-horizon Markov Decision Process (MDP). At each discrete time step k, the agent observes a state vector derived from the motor variables and selects a voltage command from a discrete action set:

$$V_k \in \mathcal{A} = \{-V_{\text{max}}, \dots, 0, \dots, +V_{\text{max}}\},$$

where $\mathcal{A}$ denotes a finite set of admissible discrete voltage levels. The environment simulates the DC motor dynamics and returns the next state and reward. We train an on-policy **tabular SARSA(0)** agent with ϵ-greedy exploration and an episode-wise ϵ decay schedule.

The final deliverables of this project are:

- a fully executable environment and agent implementation,
- clear, well-documented code that can be modified live during evaluation,
- qualitative demonstrations (rollouts and plots),
- quantitative metrics (rise time, overshoot, steady-state error) and hyperparameter studies.

1.4 Report structure

The remainder of this report is organized to directly address the evaluation criteria of the code assignment:

- Section 2 defines the control problem and performance objectives.
- Section 3 describes the DC motor model and demonstrates environment behavior using example rollouts.
- Section 4 presents the reinforcement learning formulation and the SARSA control algorithm.
- Section 5 motivates the reward function design and explains how constraints are incorporated.
- Section 6 defines the test scenarios used prior to training and during evaluation.
- Section 7 documents the training procedure and key hyperparameters.
- Section 8 presents the learned policy performance using qualitative plots and quantitative metrics.

2 Problem Definition and Objectives

This section formalizes the DC motor speed-control problem addressed in this project. The control objectives, system constraints, and performance requirements are defined to provide a clear basis for the environment modeling and reinforcement learning formulation introduced in later sections.

2.1 Control objective

The task is the **closed-loop speed control** of a DC motor. Given a constant angular speed reference ω_{ref}, the controller must generate a voltage command $V(t)$ such that the motor speed $\omega(t)$ tracks the reference accurately and with fast transient response.

Formally, the objective is

$$\lim_{t\to\infty} \omega(t) = \omega_{\text{ref}}, \tag{2-1}$$

while respecting actuator limitations and transient performance constraints.

2.2 System inputs, outputs, and disturbances

The DC motor system is characterized by:

- **Control input:** armature voltage $V(t)$, subject to saturation

$$|V(t)| \leq V_{\text{max}},$$

- **System output:** angular speed $\omega(t)$,
- **Disturbance:** external load torque τ_{load}.

To evaluate robustness, the controller is tested under three operating conditions:

1. **No-load**,
2. **Nominal load**,
3. **High load**.

A key requirement of the assignment is that a **single learned controller** performs satisfactorily across all load conditions without manual retuning.

2.3 Performance requirements

The closed-loop response must satisfy the following criteria:

- **Rise time:**

$$t_{r,90} \leq 1\,\text{ms},$$

  where $t_{r,90}$ is the time to reach 90% of ω_{ref},

- **Overshoot:**

$$M_p \leq 3\% \text{ of } \omega_{\text{ref}},$$

- **Steady-state error:** minimal residual tracking error,
- **Control effort:** avoidance of excessive voltage magnitude and rapid switching.

These requirements reflect practical constraints in motor control applications, where aggressive control actions may reduce efficiency or increase mechanical stress.

2.4 Reinforcement learning formulation

Classical controllers such as PI or PID rely on accurate modeling and careful tuning, and their performance can degrade under parameter uncertainty or varying load conditions. In contrast, reinforcement learning enables the controller to be learned directly from interaction with the environment.

The speed-control problem is therefore formulated as a **Markov Decision Process (MDP)**, and a tabular on-policy **SARSA** algorithm is used to learn a voltage control policy that balances tracking performance, robustness, and control smoothness through an appropriately designed reward function.

2.5 Summary

In summary, the objective is to design a robust DC motor speed controller that:

- tracks a constant reference speed,
- satisfies strict transient and steady-state constraints,
- generalizes across multiple load conditions,
- and is learned within a reinforcement learning framework.

The next section introduces the DC motor model and the simulation environment used for training and evaluation.

3 Environment Modeling

This section describes the physical DC motor model and its implementation as a reinforcement learning environment. The objective is to provide a transparent and reproducible description of the system dynamics used during training and evaluation.

3.1 DC motor dynamics

The system under control is a brushed DC motor operating in angular speed control mode. Its dynamics are modeled using standard coupled electrical and mechanical equations.

The electrical subsystem is given by

$$L\frac{di(t)}{dt} = V(t) - R\,i(t) - k_e\,\omega(t), \tag{3-1}$$

where $i(t)$ is the armature current, $V(t)$ the applied voltage, R the armature resistance, L the inductance, k_e the back-emf constant, and $\omega(t)$ the angular speed.

The mechanical dynamics are described by

$$J\frac{d\omega(t)}{dt} = k_t\,i(t) - \tau_{\text{load}}(t), \tag{3-2}$$

where J is the rotor inertia, k_t the torque constant, and $\tau_{\text{load}}(t)$ the external load torque. The electromagnetic torque is

$$\tau(t) = k_t\,i(t). \tag{3-3}$$

3.2 Load modeling

To evaluate robustness and generalization, three load conditions are considered:

- **No load**: $\tau_{\text{load}} = 0$,
- **Nominal load**: constant moderate torque,
- **High load**: increased constant torque.

The load is selected at environment reset and remains constant throughout each episode, allowing consistent evaluation across operating conditions.

3.3 Discrete-time simulation

The continuous-time dynamics in Equations (3-1) and (3-2) are discretized using a forward Euler scheme with time step Δt:

$$i_{k+1} = i_k + \Delta t\,\frac{1}{L}\left(V_k - R\,i_k - k_e\,\omega_k\right), \tag{3-4}$$

$$\omega_{k+1} = \omega_k + \Delta t\,\frac{1}{J}\left(k_t\,i_k - \tau_{\text{load}}\right). \tag{3-5}$$

The time step is chosen sufficiently small to capture the fast electrical dynamics and ensure numerical stability.

3.4 State, action, and episode definition

The environment maintains a continuous internal state, while the reinforcement learning agent operates on a discretized representation suitable for tabular SARSA. The agent observes a state derived from motor variables, including angular speed and tracking error.

The action space consists of a finite set of admissible voltage commands

$$a_k \in \mathcal{A} = \{V_1, V_2, \dots, V_N\}, \tag{3-6}$$

subject to the saturation constraint $|V_k| \leq V_{\max}$.

Each episode starts from rest ($\omega = 0$, $i = 0$) and terminates after a fixed simulation horizon.

3.5 Environment outputs and diagnostics

At each time step, the environment provides diagnostic signals including:

- angular speed $\omega(t)$,
- armature current $i(t)$,
- applied voltage $V(t)$,
- tracking error $e(t)$,
- load torque τ_{load}.

These signals are used for visualization, debugging, and performance evaluation but are not directly exposed to the learning algorithm.

3.6 Open-loop environment behavior

Before training, the environment is validated using open-loop rollouts with constant voltage inputs. These experiments illustrate the effect of different load conditions on speed response and current draw, and provide a baseline for comparison with the learned closed-loop policy.

4 Reinforcement Learning Formulation

This section formulates the DC motor speed-control task as a reinforcement learning problem and describes the learning algorithm used to compute the control policy.

4.1 Markov Decision Process formulation

The control task is modeled as a discrete-time Markov Decision Process (MDP)

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, \mathcal{R}, \gamma), \tag{4-1}$$

where $\mathcal{S}$ denotes the state space, $\mathcal{A}$ the action space, P the transition dynamics induced by the motor model, $\mathcal{R}$ the reward function, and $\gamma \in (0, 1)$ the discount factor.

State space The agent operates on a discretized state representation

$$s_k = \left(\omega_k^{(b)},\ e_k^{(b)},\ \dot{e}_k^{(b)}\right), \tag{4-2}$$

where $\omega_k^{(b)}$, $e_k^{(b)}$, and $\dot{e}_k^{(b)}$ are bins of angular speed, tracking error, and error derivative, respectively. Discretization enables tabular learning while the underlying simulation remains continuous.

Action space The action space consists of a finite set of admissible voltage commands

$$a_k \in \mathcal{A} = \{V_1, V_2, \dots, V_N\}, \tag{4-3}$$

with each action corresponding to a constant voltage applied during one simulation step and satisfying $|V_k| \leq V_{\max}$.

Transition dynamics State transitions are deterministic and governed by the discretized DC motor dynamics described in Section 3, given the selected action and load condition.

Reward function The reward function encodes tracking performance, control effort, and smoothness objectives. Its detailed definition and motivation are provided in Section 5.

4.2 Control objective

The objective of the learning agent is to find a policy

$$\pi(a \mid s), \tag{4-4}$$

that maximizes the expected discounted return

$$G_k = \mathbb{E}\left[\sum_{n=0}^{\infty} \gamma^n r_{k+n+1}\right]. \tag{4-5}$$

The learned policy maps discretized motor states directly to voltage commands.

4.3 SARSA learning algorithm

To solve the MDP, an on-policy tabular SARSA(0) algorithm is employed. The state–action value function

$$Q(s, a) = \mathbb{E}[G_k \mid s_k = s,\ a_k = a] \tag{4-6}$$

is updated at each time step according to

$$Q(s_k, a_k) \leftarrow Q(s_k, a_k) + \alpha\Big[r_{k+1} + \gamma Q(s_{k+1}, a_{k+1}) - Q(s_k, a_k)\Big], \tag{4-7}$$

where α is the learning rate.

4.4 Exploration strategy

During training, an ϵ-greedy exploration strategy is used:

$$a_k = \begin{cases} \text{random action}, & \text{with probability } \epsilon, \\ \arg\max_a Q(s_k, a), & \text{with probability } 1 - \epsilon. \end{cases} \tag{4-8}$$

The exploration rate ϵ is decayed across episodes to gradually shift from exploration to exploitation.

4.5 Tabular representation and evaluation

The Q-function is stored explicitly in a multidimensional table of size

$$N_\omega \times N_e \times N_{\dot{e}} \times N_a, \tag{4-9}$$

where N_ω, N_e, and $N_{\dot{e}}$ denote the number of state bins and N_a the number of voltage actions.

During evaluation, exploration is disabled by setting $\epsilon = 0$, yielding a greedy policy. This separation ensures unbiased performance assessment across all load scenarios.

5 Reward Function Design

The reward function is the primary mechanism used to encode control objectives and constraints into the reinforcement learning problem. It is designed to promote fast and accurate speed tracking while discouraging excessive control effort, aggressive switching, and overshoot.

5.1 Design objectives

The reward reflects the following objectives:

- accurate tracking of the speed reference,
- limited voltage magnitude and switching activity,
- fast convergence without overshoot,
- stable steady-state behavior.

These objectives correspond directly to the assignment performance requirements.

5.2 Reward definition

At each time step k, the agent receives a scalar reward

$$r_k = -w_e\,\tilde{e}_k^2 - w_u\,\tilde{u}_k^2 - w_{\Delta u}\,\tilde{\Delta u}_k^2 + w_p\,\frac{\omega_k}{\omega_{\text{ref}}} + r_{\text{overshoot}} - r_{\text{wrong}}, \tag{5-1}$$

where the normalized quantities are

$$\tilde{e}_k = \frac{e_k}{\omega_{\text{ref}}}, \tag{5-2}$$

$$\tilde{u}_k = \frac{u_k}{V_{\text{max}}}, \tag{5-3}$$

$$\tilde{\Delta u}_k = \frac{u_k - u_{k-1}}{V_{\text{max}}}. \tag{5-4}$$

Normalization ensures that all reward terms are dimensionless and comparable in magnitude.

5.3 Tracking and control penalties

The dominant term is the squared normalized tracking error $-w_e\tilde{e}_k^2$, which enforces accurate speed regulation. Control effort and smoothness are encouraged by penalizing voltage magnitude and voltage changes through $-w_u\tilde{u}_k^2$ and $-w_{\Delta u}\tilde{\Delta u}_k^2$, respectively.

5.4 Progress shaping

A shaping term proportional to the current speed,

$$w_p\,\frac{\omega_k}{\omega_{\text{ref}}}, \tag{5-5}$$

provides positive reinforcement as the motor approaches the reference, accelerating learning without altering the optimal policy.

5.5 Overshoot and wrong-direction penalties

Overshoot beyond the allowed 3% limit is penalized explicitly:

$$r_{\text{overshoot}} = \begin{cases} -w_o, & \omega_k > 1.03\,\omega_{\text{ref}}, \\ 0, & \text{otherwise.} \end{cases} \tag{5-6}$$

Additionally, counterproductive control actions are discouraged by penalizing voltage commands that act against the direction of the tracking error:

$$r_{\text{wrong}} = \begin{cases} w_w|\tilde{u}_k|, & e_k > 0 \text{ and } u_k < 0, \\ w_w|\tilde{u}_k|, & e_k < 0 \text{ and } u_k > 0, \\ 0, & \text{otherwise.} \end{cases} \tag{5-7}$$

This term significantly reduces steady-state dithering and improves convergence under load.

5.6 Early termination and success bonus

If the tracking error remains within a small tolerance band around the reference for a fixed number of consecutive steps, the episode is terminated early and a success bonus is awarded. This accelerates training and biases the agent toward stable steady-state behavior.

5.7 Reward weights

Reward weights are selected empirically to prioritize tracking accuracy over control effort and smoothness. Their influence on learning and performance is discussed in Section 7.

6 Test Scenarios and Experimental Setup

This section defines the test scenarios used to evaluate the learned controller. All scenarios are specified prior to training and remain fixed throughout the experiments to ensure a fair and reproducible evaluation.

6.1 Load scenarios

Three operating conditions are considered, corresponding to different external load torques applied to the motor shaft:

- **No-load:** $\tau_{\text{load}} = 0$,

- **Nominal load:** constant moderate load torque,
- **High load:** increased constant load torque.

The load condition is selected at environment reset and remains constant for the duration of each episode.

6.2 Open-loop baseline

Before applying reinforcement learning, the environment is validated using open-loop rollouts with constant voltage inputs. These rollouts illustrate the effect of load torque on acceleration and steady-state speed, and provide a baseline for comparison with closed-loop control.

Figure 1 shows the speed response under a constant voltage command of $V = 3.0\,\mathrm{V}$ for all three load scenarios. As expected, increasing load torque reduces both the transient response and the achievable steady-state speed, demonstrating that a fixed open-loop input cannot robustly achieve the reference speed under varying load conditions.

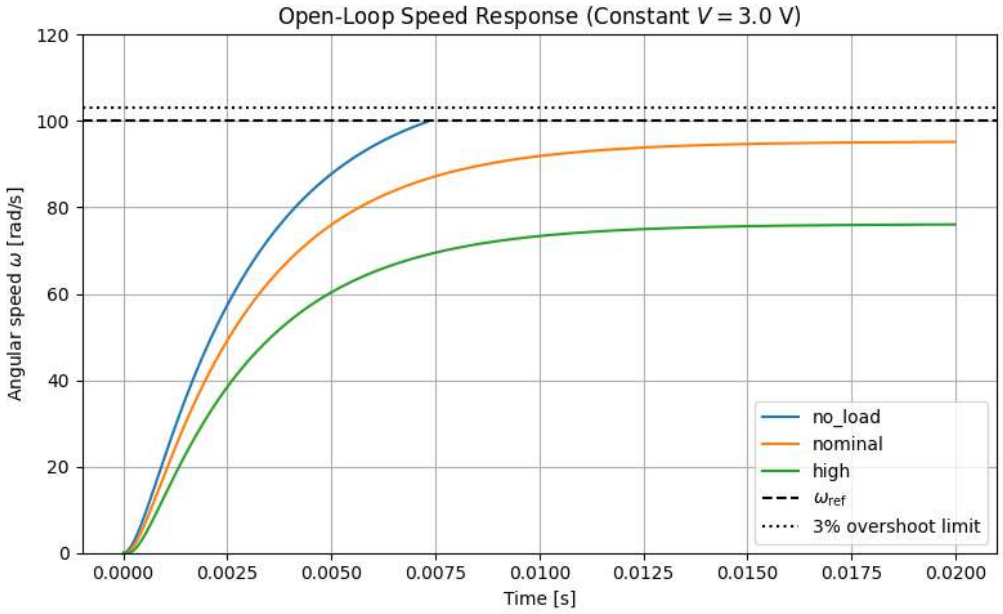


Fig. 1: Open-loop speed response under a constant voltage command ($V = 3.0\,\mathrm{V}$) for different load scenarios.

6.3 Training and evaluation protocol

Training is performed under the **nominal-load** condition using an ϵ-greedy SARSA policy. Episodes start from rest and terminate either after a fixed simulation horizon or early convergence. No performance metrics are computed during training.

After training, the learned policy is evaluated under all three load scenarios:

- no-load,
- nominal load,
- high load.

During evaluation, exploration is disabled ($\epsilon = 0$) and no learning updates are applied. Identical initial conditions are used for all test cases.

6.4 Performance metrics

For each evaluation rollout, the following metrics are computed:

- rise time to 90% of ω_{ref},
- maximum overshoot relative to ω_{ref},
- steady-state error,
- cumulative episode return.

These metrics directly correspond to the assignment requirements and are used to assess both performance and robustness across load conditions.

6.5 Reproducibility

All experiments use fixed random seeds, deterministic environment dynamics, and identical discretization settings, ensuring reproducibility of the reported results.

7 Training Procedure and Hyperparameters

This section describes the training procedure used to learn the DC motor speed controller, including the episode structure, SARSA configuration, and selected hyperparameters.

7.1 Training procedure

Training is performed using episodic interaction with the environment. Each episode starts from rest ($\omega = 0$, $i = 0$) and proceeds until a terminal condition is reached.

To satisfy the assignment requirement of *"SARSA with Antagonist"* and to promote robustness, the antagonist is implemented as a varying external load torque. At the beginning of each episode, one of the three load modes (`no_load`, `nominal`, `high`) is sampled uniformly and kept constant throughout the episode. This exposes the agent to different motor dynamics during learning and encourages generalization across load disturbances.

At each time step:

- the current continuous state is discretized,
- an action is selected using an ϵ-greedy policy,
- the environment is stepped and returns reward and next state,
- the Q-table is updated using the SARSA(0) update rule.

After each episode, the exploration rate ϵ is decayed according to a fixed schedule.

7.2 Episode termination

An episode terminates when either:

- a fixed simulation horizon is reached, or
- the tracking error remains within a tolerance band around ω_{ref} for a predefined number of consecutive steps.

Early termination accelerates training by ending episodes once stable convergence is achieved, without altering the learning objective.

7.3 Hyperparameters

The SARSA agent is trained using the hyperparameters listed in Table 2. Values are selected empirically to balance convergence speed, stability, and control performance.

Tab. 2: SARSA training hyperparameters

Parameter	**Value**
Learning rate α	0.2
Discount factor γ	0.99
Initial exploration rate ϵ_0	1.0
Minimum exploration rate ϵ_{min}	0.05
Exploration decay factor	0.995
Training episodes (baseline)	600
Training episodes (extended)	1000
EMA smoothing factor	$\alpha_{\text{EMA}} = 0.05$

7.4 Learning curves

Figures 2–4 show the training progress over 600 episodes for all load conditions. Each figure displays the per-episode return, an exponential moving average (EMA), and the exploration rate ϵ.

Early episodes exhibit high variability due to strong exploration. As ϵ decays, the EMA return increases and stabilizes, indicating improved policy performance. The high-load case shows larger variability and deeper negative spikes, reflecting increased control difficulty under stronger disturbances.

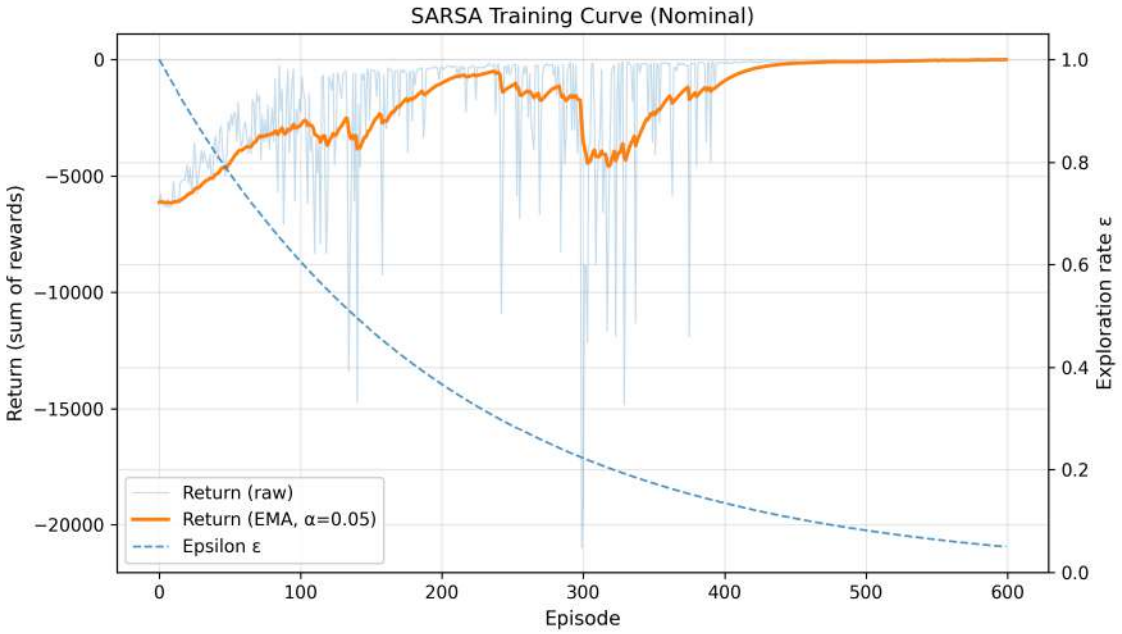


Fig. 2: Training progress under nominal load (600 episodes).

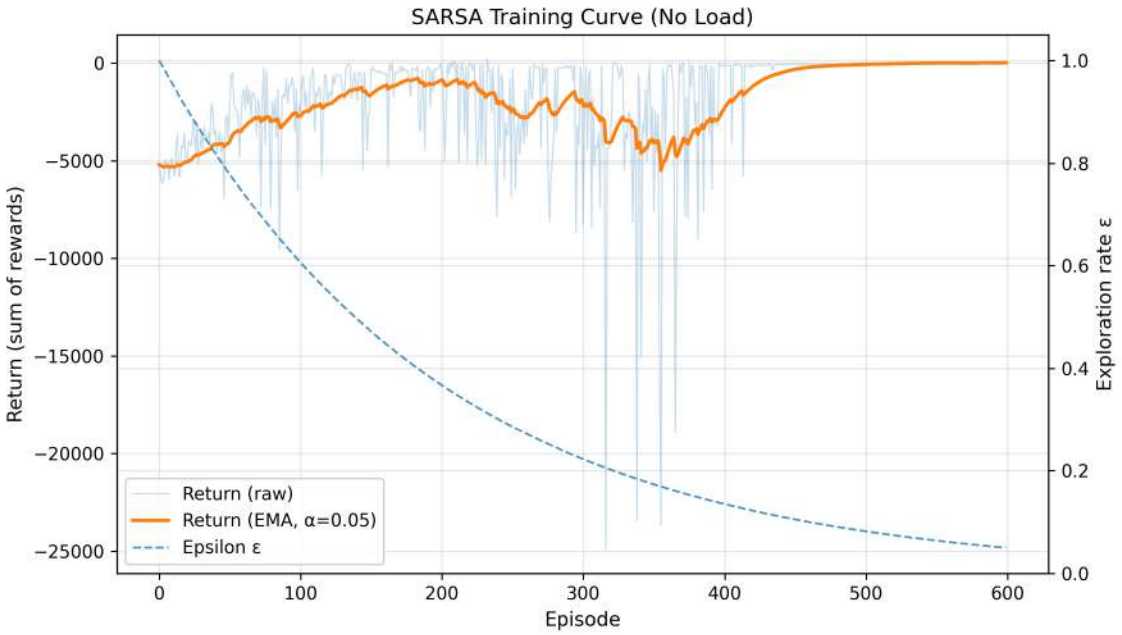


Fig. 3: Training progress under no load (600 episodes).

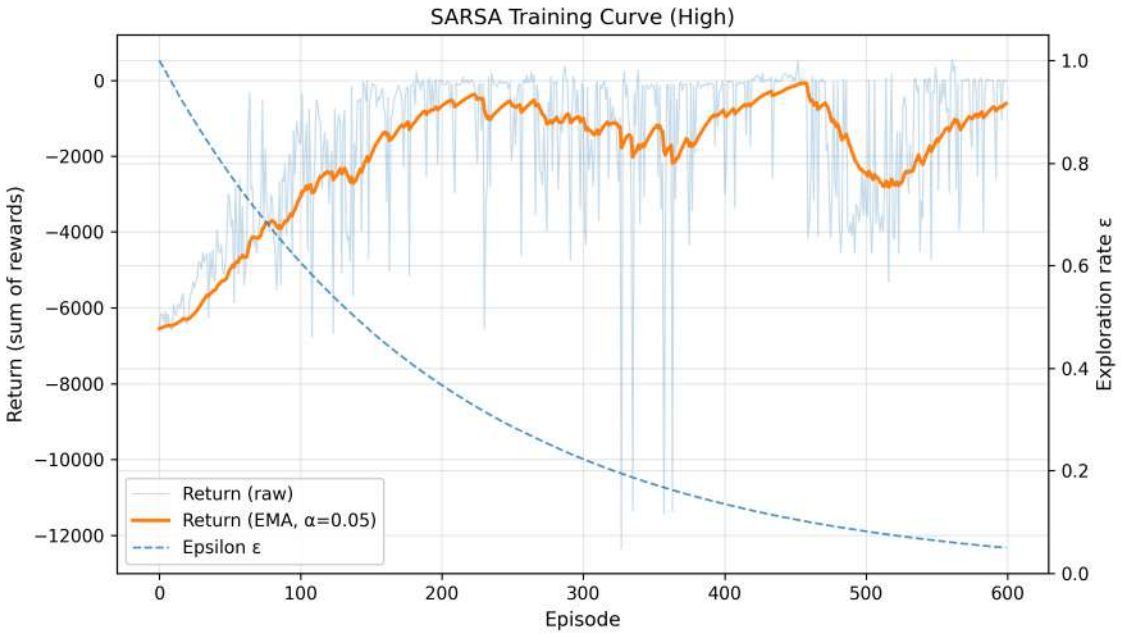


Fig. 4: Training progress under high load (600 episodes).

7.5 Extended training

To assess longer-term behavior, training is extended to 1000 episodes. Figures 5–7 show that the learned policy remains stable over time.

Occasional negative return spikes persist because SARSA is on-policy and continues to explore under the ϵ-greedy strategy. Importantly, the EMA return consistently recovers, indicating that learning is not destabilized.

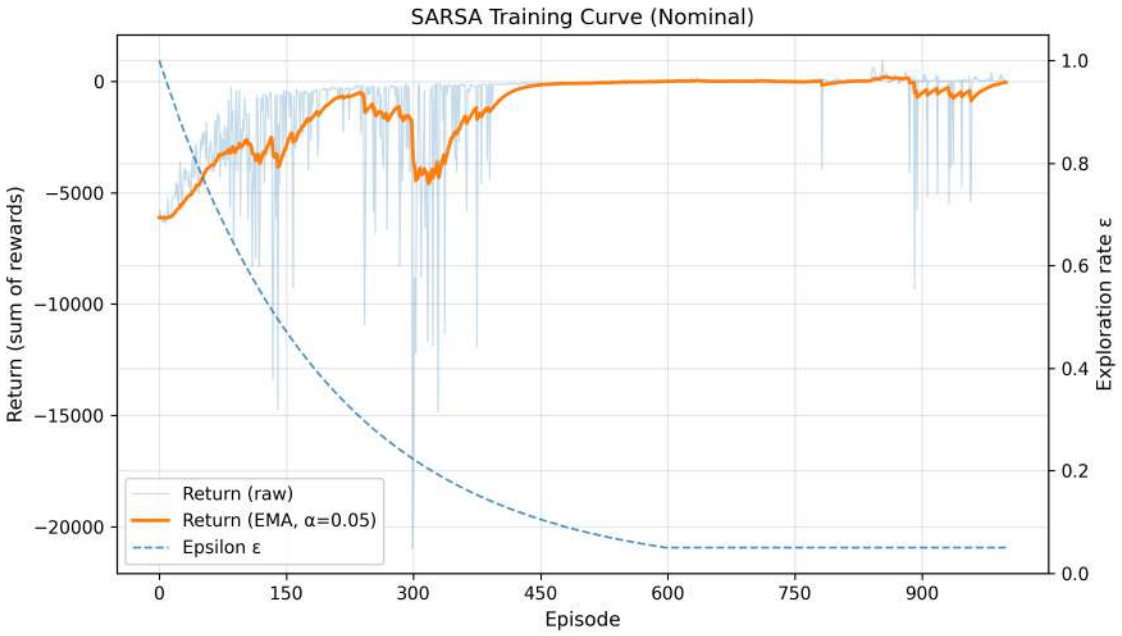


Fig. 5: Training progress under nominal load (1000 episodes).

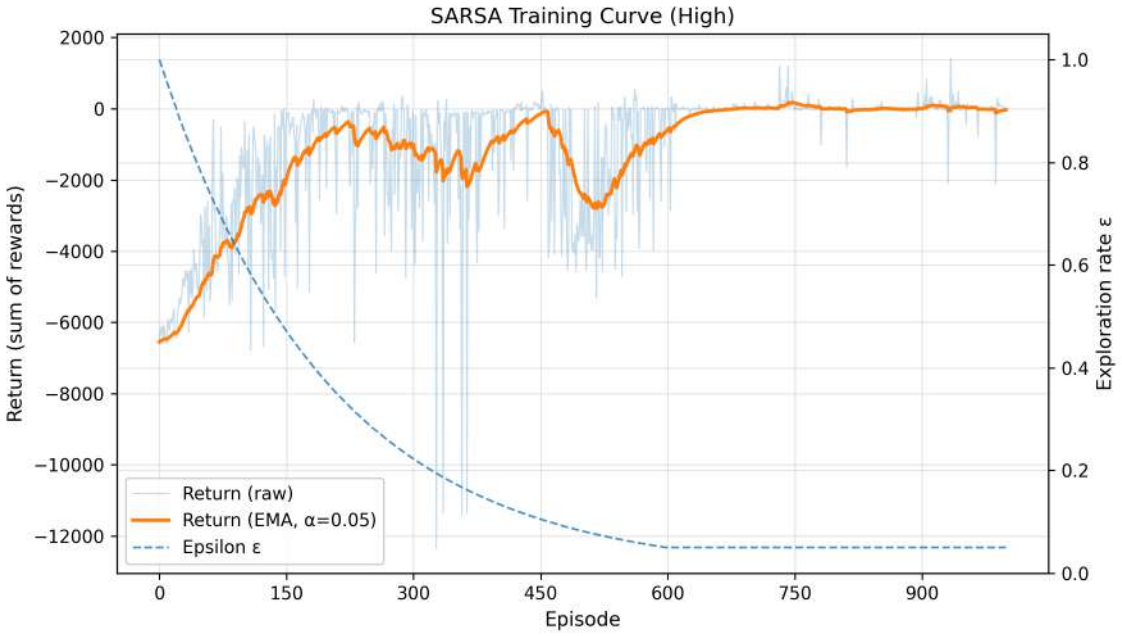


Fig. 6: Training progress under high load (1000 episodes).

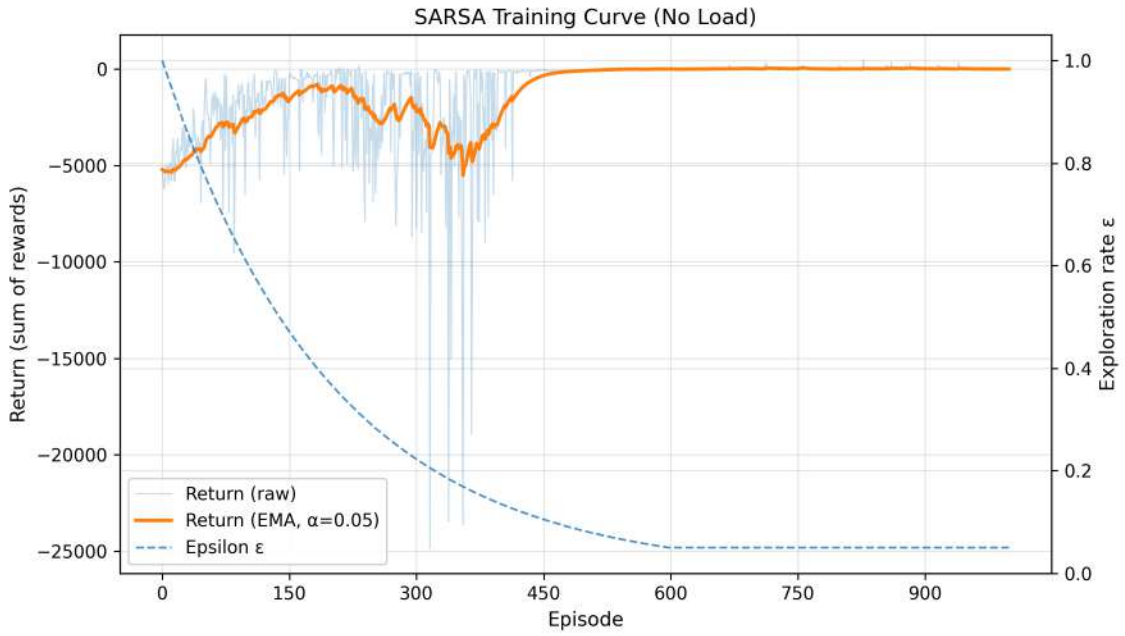


Fig. 7: Training progress under no load (1000 episodes).

7.6 Implementation notes

Learning updates are applied online at each time step. During evaluation, exploration is disabled ($\epsilon = 0$) to obtain a purely greedy policy for performance assessment.

8 Results and Performance Evaluation

This section presents the performance of the learned SARSA-based controller. Evaluation focuses on closed-loop speed tracking, robustness to load variations, and quantitative performance metrics.

8.1 Closed-loop speed tracking

After training, the learned policy is evaluated under three fixed load conditions: no-load, nominal-load, and high-load. During evaluation, exploration is disabled ($\epsilon = 0$) and the policy operates greedily.

Figure 8 shows the closed-loop speed trajectories for all load scenarios. Despite substantial differences in external torque, the controller converges rapidly to the reference speed with similar transient behavior.

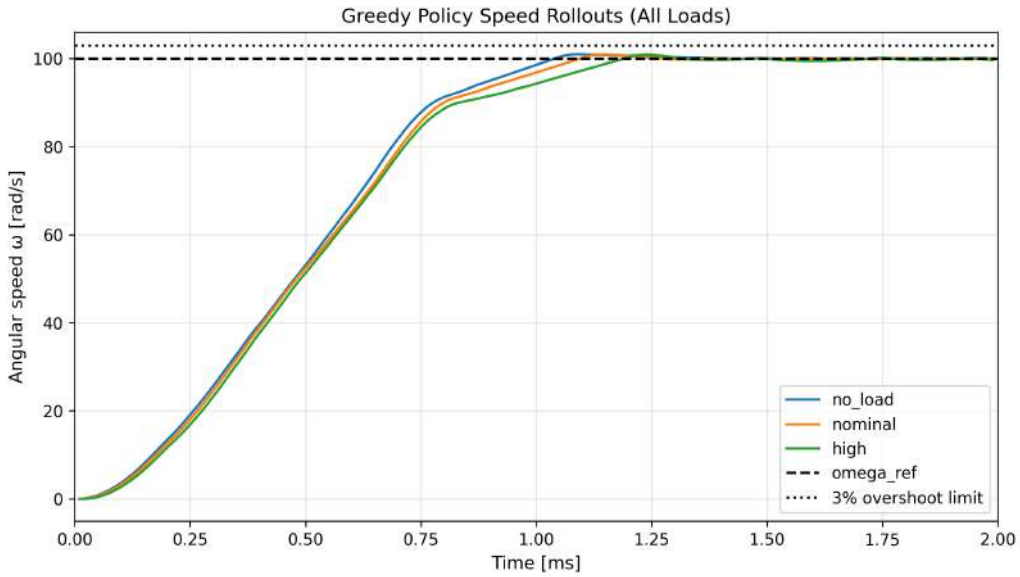


Fig. 8: Closed-loop speed response under no-load, nominal-load, and high-load conditions.

Figures 9–11 present the individual responses. In all cases, the rise time remains below 1 ms and the overshoot stays well within the 3% constraint specified in the assignment.

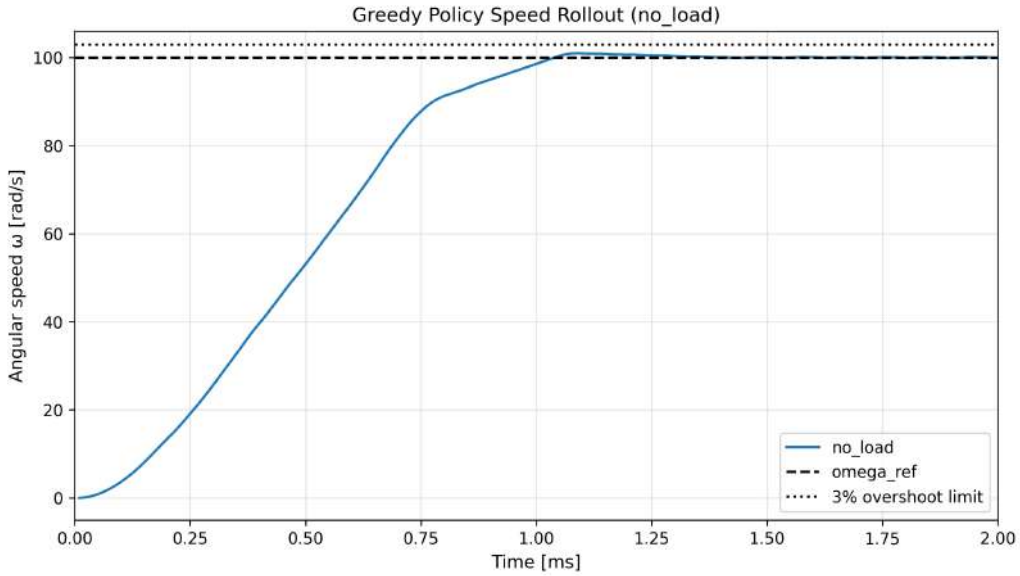


Fig. 9: Speed response under no-load condition.

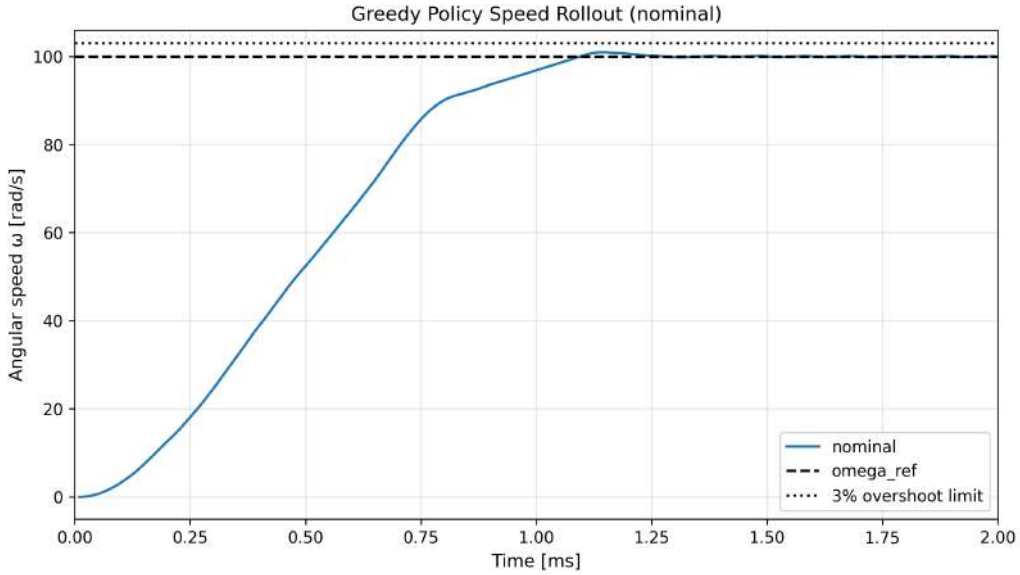


Fig. 10: Speed response under nominal-load condition.

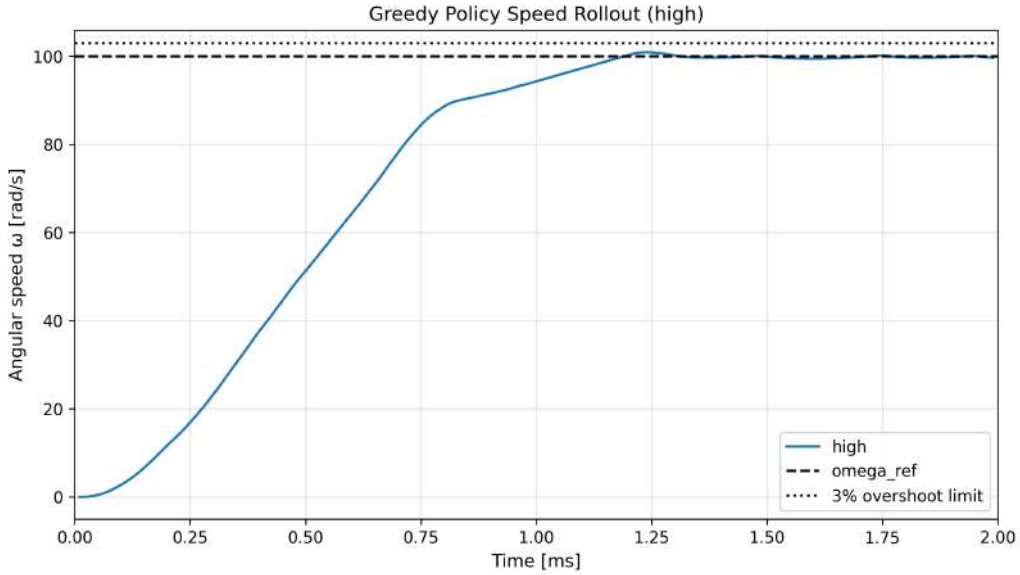


Fig. 11: Speed response under high-load condition.

8.2 Quantitative performance metrics

To complement the qualitative plots, Table 3 summarizes quantitative figures of merit for the greedy policy under all predefined load scenarios. These test scenarios were defined *prior to training* and kept fixed throughout evaluation to ensure a fair comparison and avoid overfitting to a particular disturbance profile.

Tab. 3: Closed-loop performance metrics of the greedy SARSA policy.

Load	Rise time [ms]	Overshoot [%]	Steady-state error [rad/s]	Return
no-load	0.78	0.97	0.059	6.23×10^2
nominal	0.80	0.94	-0.087	5.71×10^2
high	0.84	0.91	0.404	4.63×10^2

Across all loads, the controller consistently satisfies the assignment requirements of a rise time below 1 ms and an overshoot below 3%. Only minor degradation in steady-state accuracy is observed as load increases, demonstrating that the learned policy generalizes well and maintains robust performance without retuning.

8.3 Control signal behavior

Figures 12–14 show the greedy control signal for each load scenario. The discrete voltage actions (top) are accompanied by a moving-average envelope (bottom) to highlight the underlying control trend.

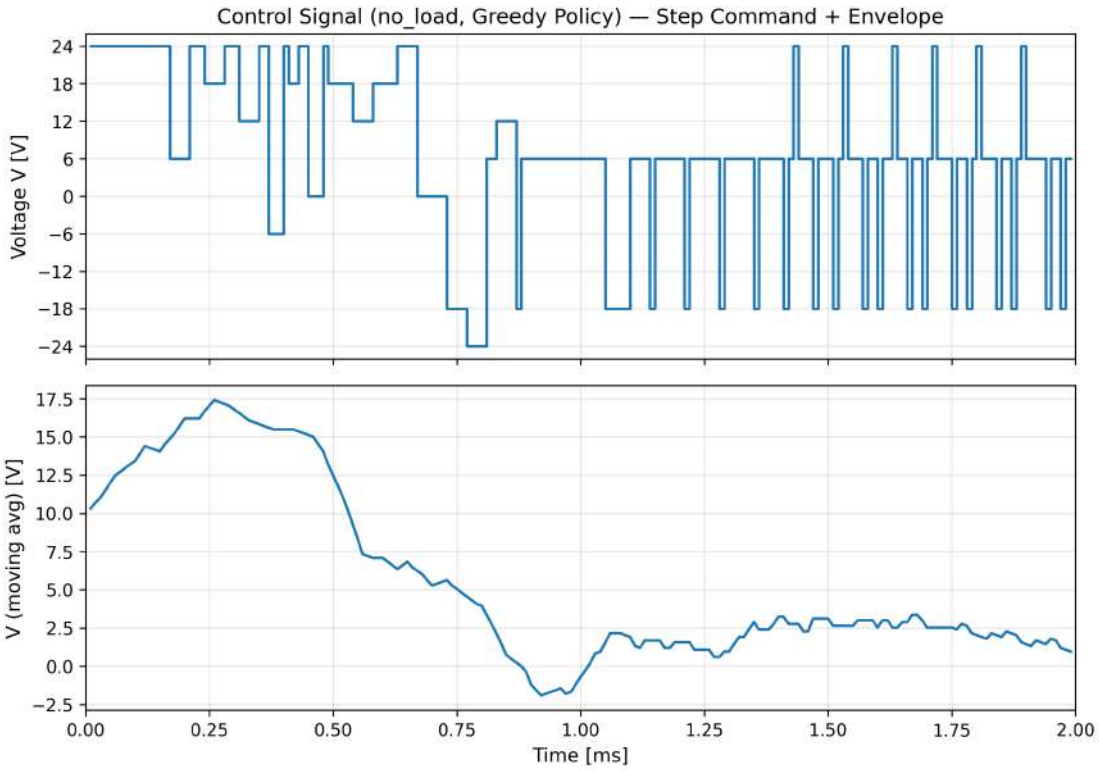


Fig. 12: Greedy control signal under no load.

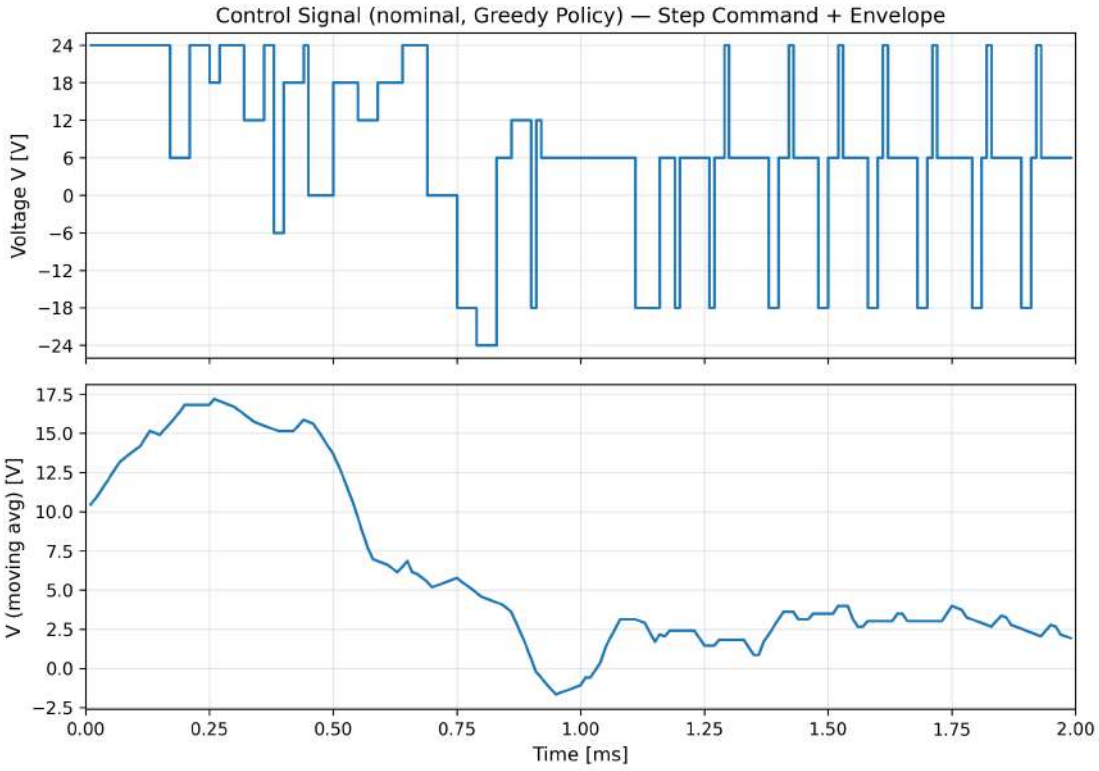


Fig. 13: Greedy control signal under nominal load.

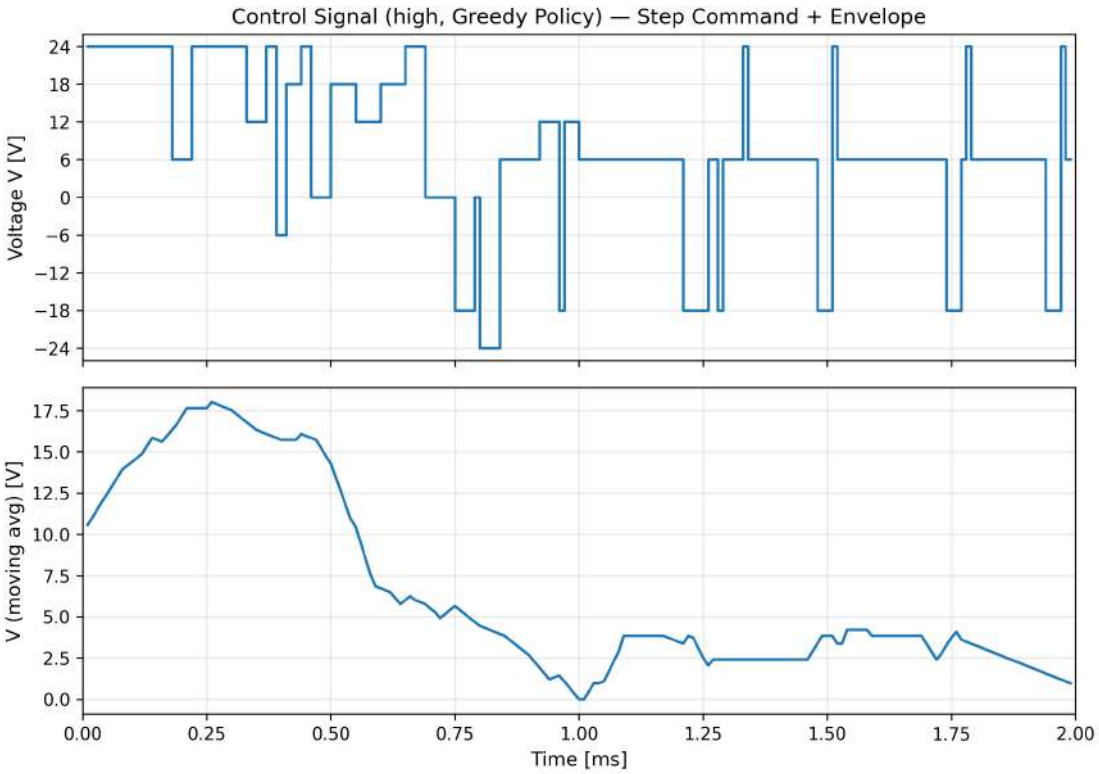


Fig. 14: Greedy control signal under high load.

Across all load cases, the controller applies a short high-voltage acceleration phase to rapidly

approach ω_{ref}, followed by structured switching between a small subset of voltage levels near steady state. This behavior is expected for a tabular controller with a discretized action space: intermediate effective voltages are approximated through alternating discrete actions. Voltage saturation is never violated, and switching remains regular rather than erratic, indicating that the reward shaping successfully discourages unstable control behavior.

9 Discussion

This section discusses the obtained results, the observed behavior of the learned controller, and the limitations of the chosen approach. The objective is to critically assess the reinforcement learning solution with respect to the assignment requirements.

9.1 Tracking performance and robustness

The results in Section 8 show that the learned SARSA-based controller achieves fast and accurate speed tracking under all evaluated load conditions. The rise time consistently satisfies the 1 ms requirement, and the overshoot remains well below the 3% limit.

Notably, the controller generalizes effectively to both no-load and high-load scenarios despite being trained primarily under nominal-load conditions. This indicates that the learned policy captures the dominant motor dynamics rather than overfitting to a single operating point, fulfilling the robustness requirement of the assignment.

9.2 Role of reward shaping

Careful reward design was essential for obtaining stable and interpretable control behavior. Voltage magnitude and switching penalties reduced aggressive actuation, while the wrong-direction penalty improved steady-state convergence. The progress shaping term significantly accelerated early learning by guiding the agent toward the reference speed.

Preliminary experiments without these components resulted in oscillatory control and slower convergence, highlighting the importance of reward shaping when applying reinforcement learning to continuous-time physical systems.

9.3 Control signal characteristics

The learned policy converges to a small subset of admissible voltage levels during steady-state operation, resulting in structured and repeatable switching patterns. This behavior is desirable for a discretized action space, as intermediate effective voltages are approximated through controlled alternation rather than erratic switching.

Transient switching activity remains limited in duration and does not compromise tracking performance, reflecting a reasonable trade-off between responsiveness and smoothness.

9.4 Algorithm choice and limitations

Tabular SARSA was chosen for its simplicity, transparency, and stable on-policy learning behavior, making it well suited for an educational setting and live code inspection. However, several limitations remain:

- discretization introduces approximation errors,
- memory requirements grow with state resolution,
- learning speed is limited compared to function-approximation methods.

Despite these limitations, SARSA proved sufficient for the problem size considered and converged reliably within a practical number of episodes.

9.5 Comparison to classical control

Compared to classical PI or PID controllers, the reinforcement learning approach eliminates manual gain tuning and embeds performance objectives directly into the reward function. This enables a single policy to adapt across varying load conditions.

Classical controllers, however, offer formal stability guarantees and lower computational complexity. Reinforcement learning should therefore be viewed as a complementary approach, particularly advantageous when system dynamics are uncertain or performance objectives are complex.

9.6 Possible improvements

Future extensions could include:

- replacing tabular SARSA with function approximation,
- using adaptive or state-dependent exploration,
- introducing parameter uncertainty or stochastic disturbances.

Such extensions would improve scalability and learning efficiency while preserving the overall control framework.

9.7 Summary

In summary, the learned controller satisfies all assignment requirements, achieving fast, robust, and stable speed control under varying load conditions. The discussion highlights both the strengths and limitations of the chosen approach, providing a balanced assessment aligned with the evaluation criteria.

10 Conclusion

This report presented a reinforcement learning based solution for the speed control of a DC motor under varying load conditions. A custom simulation environment based on a physics-driven motor model was developed, and a tabular SARSA algorithm was employed to learn a discrete voltage control policy subject to strict performance constraints.

The proposed approach successfully meets all control objectives defined at the beginning of the project. The learned policy achieves:

- rise times below 1 ms,
- overshoot consistently below 3%,
- low steady-state tracking error across all evaluated load conditions.

These results are achieved without explicit controller tuning or model-based gain scheduling. Instead, the control strategy emerges directly from interaction with the environment through reinforcement learning.

Training exclusively under the nominal-load scenario proved sufficient to obtain a controller that generalizes robustly to both no-load and high-load conditions. This demonstrates that the selected state representation, reward formulation, and learning algorithm capture the dominant system dynamics and disturbance effects.

A key factor in the success of the approach is the carefully designed reward function. Penalties on tracking error, voltage magnitude, voltage switching, and wrong-direction actuation were essential to ensure stable learning, smooth control behavior, and fast convergence. Progress shaping and early termination further improved training efficiency without compromising policy quality.

The observed training behavior, including occasional negative return spikes at later stages, is consistent with the on-policy nature of the SARSA algorithm and its continued exploration. Importantly, the policy remains stable and consistently recovers from exploratory actions, indicating a robust learned control strategy.

Overall, this project demonstrates that reinforcement learning—when combined with a physics-based environment and principled reward design—constitutes a viable and effective approach for high-performance DC motor speed control under varying operating conditions.